ProxiCheck: Intelligent University Attendance System

Technical Architecture, Engineering Methodology & Project Report

1. Project Overview & Problem Statement

1.1 System Definition

ProxiCheck is a hardware-free, browser-native university attendance management platform engineered for academic institutions. It enables faculty members to initiate cryptographically verified roll-call geofences centered around physical classrooms and allows enrolled students to record verified attendance within seconds using standard web browsers on their existing mobile phones, tablets, or laptops.

1.2 The Problem It Solves

Traditional university attendance tracking suffers from severe structural vulnerabilities:

- Paper Sign-in Sheets: Prone to proxy sign-ins, illegible handwriting, misplaced rosters, and wasted lecture time passing sheets across large auditoriums.
- Dedicated Biometric Hardware: Fingerprint and facial terminals are cost-prohibitive to deploy across all lecture halls, suffer mechanical breakdown, and cause bottleneck lines at entrances.
- Static QR Code Systems: Flawed because students in class take screenshots of projected QR codes and forward them to group chats for absent classmates to scan from hostels.
- Native Mobile Apps & Beacons: Compelling students to download 80MB+ native app store packages causes storage and compatibility barriers, while Bluetooth beacon signals unpredictably leak through concrete walls.

ProxiCheck resolves these challenges by removing physical hardware dongles and app store barriers entirely, enforcing physical presence through browser geolocation, server-side geospatial verification, single-device fingerprint binding, and institutional domain validation.

2. Attendance Process & Verification Mechanics

2.1 Step-by-Step Attendance Workflow

- Session Launch: The lecturer logs into the Faculty Command Center, selects an assigned departmental course, picks a registered hall (e.g., Step B), sets an allowed radius (10m to 500m), and configures duration.
- GPS Anchoring: The session defaults to the calibrated coordinates of the hall, or dynamically anchors to the lecturer's live device coordinates if the class relocated.
- Session Discovery: Students authenticated with valid university credentials see the active roll call on their dashboard if officially enrolled in that course.
- Presence Submission: The student taps 'Check In'. The browser queries the device satellite hardware for high-accuracy latitude, longitude, and horizontal accuracy.
- Multi-Layer Verification: The server validates device integrity, filters coarse signals, computes distance to the hall center, and executes an atomic database transaction.
- Live Confirmation: The student receives immediate verified feedback, and the lecturer's monitor updates in real time with attendee details and matric numbers.

2.2 Verification Logic (No Raw Formulas)

The server validates attendance through four strict sequential criteria:

- Single-Device Integrity: The student's account must match the physical device

hardware fingerprint registered upon signup, preventing buddy check-ins.

- Sensor Accuracy Filter: The device's reported GPS horizontal accuracy radius must be within acceptable thresholds (under 50 meters), filtering out coarse cell-tower and Wi-Fi triangulations.
- Geodesic Distance Verification: The server computes the true shortest surface distance between the student's coordinates and the lecture hall center using the Great-Circle navigation algorithm. The student must physically be within the lecturer's configured perimeter.
- Atomic Concurrency Lock: A composite unique constraint on session ID and student ID guarantees that subsequent check-in attempts are rejected, ensuring zero duplicate submissions.

3. Technologies Used

Layer	Technology	Role & Engineering Responsibility
Frontend Framework	Next.js 15.2 (App Router)	Se
UI & Interactivity	React 19 & Tailwind CSS	Reactive live state, mobile-first responsive des
Programming Language	TypeScript 5.8 (Strict)	End-to-end type safety across API contracts and
Database & Auth	Supabase (PostgreSQL 15)	Relational data, Row-Level Security policies, an
Fallback Store	In-Memory Store (ES6 Maps)	Zero-dependency offline/demo operational databas
Testing Engine	Vitest 3 & Testing Library	Unit testing for geo-algorithms, role validation

4. Implemented Features & In-Progress Additions

4.1 Core Implemented Features

- Browser-Native Geofencing: Zero app store downloads; students check in directly via browser geolocation.
- Single-Device Hardware Binding: Links each student account to one device fingerprint, stopping proxy logins.
- Faculty Key Verification: Pre-provisioned departmental authorization keys prevent student spoofing of lecturer roles.
- Institutional Email Enforcement: Strict validation requiring students to use OAU student emails and staff to use faculty emails.
- Venue & Course Management: Full administrative management for lecture theatres and departmental allocations.
- Live Attendance Radar & Export: Real-time polling monitor and one-click formatted CSV sheet export for academic portals.
- Individual Course Enrollment: Students can only access roll calls for courses they are officially registered for.

4.2 Exam Eligibility & At-Risk Predictor Feature

To support academic regulations requiring students to attain at least 75% lecture attendance to qualify for semester examinations, ProxiCheck includes an attendance forecasting engine:

- Dynamic Attendance Projection: Calculates the student's maximum attainable attendance percentage by summing attended classes with all remaining scheduled classes, divided by total syllabus lectures.
- Automated Risk Badging: Flags students as Safe (at or above 75%), At-Risk (60% to 74% with target attendance alerts), or Barred (under 60% or mathematically unable to reach 75%).
- Official Examination Docket: Provides lecturers and examination officers with a one-click generated summary of barred students for faculty board review.

5. System Architecture, Workflows & Database Design

5.1 Architectural Tiering

ProxiCheck is built on a 3-tier cloud architecture:

- Presentation Tier: Client web application rendered via Next.js 15, handling location capture and user interaction.
- Application Tier: Next.js Edge and Serverless route handlers executing authentication guards, geospatial validation, and business logic.
- Data Tier: Supabase PostgreSQL 15 database enforcing Row-Level Security, unique attendance constraints, and relational integrity.

5.2 Relational Database Schema Entities

- profiles: User identity containing ID, email, full name, role (student/lecturer/admin), matric number, and bound device ID.
- classrooms: Physical campus venues storing venue name, building, latitude, longitude, and default radius.
- courses: Academic subjects storing course code, name, and departmental affiliation.
- course_enrollments: Relational mapping connecting students to officially enrolled courses.
- attendance_sessions: Lecture sessions tracking course ID, lecturer ID, venue, coordinates, radius, and active status.
- attendance_records: Verified check-ins recording session ID, student ID, matric number, distance, and timestamps.

6. Development Process & Major Challenges Encountered

6.1 Engineering Heritage & Pivot

The project originated as an ESP32 microcontroller Bluetooth Low Energy (BLE) beacon prototype paired with an Expo React Native mobile app. During initial field testing, the hardware approach exhibited critical flaws: BLE signal strengths fluctuated through concrete walls causing false corridor check-ins, and distributing native mobile apps caused installation friction. The team pivoted to a browser-based geospatial architecture, eliminating hardware dependencies while hardening web security.

6.2 Key Challenges & Technical Solutions

- Indoor GPS Multipath Distortion: Concrete auditoriums can degrade satellite accuracy. Solved by implementing an adaptive sensor accuracy filter and a Live Location Override enabling lecturers to anchor the geofence from their active podium position.
- Buddy Check-in Prevention: Students sharing login credentials with friends in class. Solved by binding user accounts to a unique browser hardware device fingerprint upon registration.
- Database Schema Desynchronization: Cloud migrations missing columns during initial account creation. Solved by implementing resilient fallback upsert patterns that guarantee profile persistence before session initialization.

7. Testing Performed, Actual Results & System Limitations

7.1 Automated Testing Suite (Vitest 3)

The platform is verified with 50 automated tests across 5 test suites:

- Geospatial Distance Suite (9 tests): Validated against campus coordinate benchmarks. Verified boundary conditions including zero offset, inside perimeter, boundary edges, and outside limits.
- Attendance Engine Suite (7 tests): Verified atomic check-ins, active session time

windows, accuracy rejection for coarse signals, and duplicate attempt prevention.

- Role & Email Governance (16 tests): Verified strict OAU student matriculation structure and staff email formatting.
- Route Protection Middleware (10 tests): Tested role-based redirection, cookie session extraction, and unauthorized route guards.
- UI Components Suite (8 tests): Verified rendering states, loading spinners, and interactive dialogs.

7.2 Actual Test Outcomes & Known Limitations

All 50 automated tests pass with 100% success rate. Identified system limitations include:

- Browser Permissions: Location permission is required by the browser; users who deny permission cannot record presence.
- Basement Laboratories: Underground halls without satellite line-of-sight require lecturers to expand radius presets.

8. Documentation, Source Repository & Project Heritage

Project heritage and documentation resources maintained in the repository include:

- Project Inspiration: Derived architectural paradigms from open-source mobile BLE attendance prototypes, replacing the fragile radio layer with web geospatial technology.
- README Documentation: Comprehensive user guide and operational runbook located at README.md.
- Deployment Manual: Step-by-step production setup and environment variable configuration located at docs/DEPLOYMENT.md.
- Database Migrations: Versioned SQL migration scripts defining tables, triggers, and security policies located at supabase/migrations/.